

# **A Technical Blueprint for the Development of a Real-Time AI Assistant for Blackjack and Poker**

## **Part 1: Foundational Game Theory and AI Models**

This section establishes the theoretical bedrock of the AI's decision-making engines. It dissects the mathematical and strategic principles governing optimal play in both Blackjack and Poker, transitioning from deterministic rules to the probabilistic and psychological complexities of each game.

### **Mastering Blackjack: From Deterministic Rules to Probabilistic Advantage**

Blackjack presents a unique challenge as a casino game where the odds are not static; they change as cards are dealt from the deck or shoe. An effective AI agent must first master the mathematically "solved" strategy for playing any given hand and then layer on more advanced, state-dependent techniques to gain a true statistical edge over the house.

#### **The Mathematical Core: Implementing Basic Strategy**

The foundation of any proficient Blackjack play is "Basic Strategy." This is a comprehensive set of rules that dictates the mathematically optimal action for every possible two-card hand a player can receive against every possible up-card the dealer may show.<sup>1</sup> This strategy is not based on intuition or guesswork but is the result of extensive computer simulations analyzing millions of hands to determine the play that yields the highest expected value in the long run.<sup>1</sup>

Adhering perfectly to Basic Strategy is the first and most crucial step in minimizing the casino's inherent advantage, reducing the house edge to as low as approximately 0.5%.<sup>2</sup>

The implementation of this logic within the AI agent must follow a strict order of operations, mirroring the decisions a player faces at the table.<sup>4</sup> For any given hand, the decision tree is as follows:

1. **Should I Surrender?** This option, if available, allows the player to forfeit half their bet and end the hand immediately. It is only available on the first two cards. The strategy dictates surrendering a hard 16 (but not a pair of 8s) against a dealer's 9, 10, or Ace, and a hard 15 against a dealer's 10.<sup>1</sup>
2. **Should I Split?** If the player is dealt a pair, they may have the option to split them into two separate hands, placing an additional bet equal to the original wager.<sup>5</sup> The rules for splitting are highly specific, such as always splitting Aces and 8s, but never splitting 5s or 10s.<sup>1</sup>
3. **Should I Double Down?** This allows the player to double their initial bet in exchange for receiving exactly one more card.<sup>2</sup> Basic Strategy identifies specific advantageous situations for doubling, such as holding a total of 11 against any dealer up-card except an Ace.<sup>1</sup>
4. **Should I Hit or Stand?** If none of the above options are available or advisable, the final decision is to either take another card (Hit) or end the turn (Stand). This decision depends on the player's total and the dealer's up-card, such as always standing on a hard 17 or higher, or hitting on 12-16 when the dealer shows a 7 or higher.<sup>1</sup>

A critical aspect of the AI's design is its ability to adapt this strategy to the specific rules of the game being played. Basic Strategy is not a monolithic doctrine; it varies based on table rules that affect the game's probabilities.<sup>4</sup> The agent's logic must be configurable for key variations, including:

- **Number of Decks:** The game can be played with anywhere from one to eight decks.
- **Dealer's Soft 17 Rule:** Whether the dealer must Hit on a soft 17 (H17) or Stand on a soft 17 (S17) significantly alters several strategic decisions.<sup>3</sup> The H17 rule is generally more common.<sup>4</sup>
- **Doubling Rules:** Restrictions on doubling, such as only on 9, 10, or 11, or whether Doubling After a Split (DAS) is allowed, require adjustments.<sup>2</sup>
- **Surrender:** The availability of Late Surrender (LS) introduces the first decision point in the strategy hierarchy.<sup>4</sup>

The fact that casinos do not prohibit the use of Basic Strategy charts, and in some cases even sell them, reveals a fundamental truth about the game.<sup>1</sup> The casino's business model is built upon a guaranteed long-term statistical advantage. Basic Strategy only reduces this advantage to its theoretical minimum; it does not eliminate it.<sup>2</sup> The house still retains an edge of about 0.5%. The casino profits from this small edge over millions of hands played, and

more substantially, from the vast majority of players who do not adhere to perfect Basic Strategy. Players make decisions based on emotion, superstition, or common but incorrect assumptions, such as always assuming the dealer's down card is a 10 or refusing to hit a 16 against a dealer's 10 for fear of busting.<sup>2</sup> These deviations from optimal play significantly increase the house edge. Therefore, for the AI agent, Basic Strategy serves as a crucial defensive baseline—a method to prevent unnecessary losses, but not a tool to achieve a positive expectation. To cross the threshold from minimizing losses to generating profit, a more advanced strategy is required.

## Gaining an Edge: Card Counting Systems

To overcome the final 0.5% house edge and gain a positive expectation, the AI agent must employ card counting. Card counting is a technique used to track the ratio of high-value cards (Aces, 10s, Jacks, Queens, Kings) to low-value cards (2s through 6s) that have been dealt from the shoe.<sup>8</sup> The underlying principle is that a deck rich in high cards is advantageous to the player.<sup>8</sup> This is due to several factors:

- It increases the probability of the player receiving a Blackjack (an Ace and a 10-value card), which typically pays out at 3:2 odds.<sup>8</sup>
- It increases the probability of the dealer busting (exceeding 21), as the dealer must follow fixed rules for hitting on totals like 16.<sup>11</sup>
- It makes doubling down more profitable, as the player is more likely to receive a 10 on a total of 10 or 11.<sup>11</sup>

The AI will implement a card counting system, such as the widely used **Hi-Lo** system. This is a "Level 1" balanced system, valued for its relative simplicity and effectiveness.<sup>8</sup> The implementation involves three core steps:

1. **Assigning Card Values and Maintaining a Running Count:** Each card seen is assigned a tag:
  - Cards 2-6: +1
  - Cards 7-9: 0
  - Cards 10-Ace: -1As cards are dealt to all players and the dealer, the AI will update a cumulative "running count".<sup>10</sup> A positive running count indicates that more low cards than high cards have been removed from the shoe, leaving a higher concentration of advantageous high cards.
2. **Calculating the True Count:** The running count alone is insufficient. A running count of +5 is far more significant with only one deck remaining than with six decks remaining.<sup>10</sup> To normalize for this, the AI must calculate the "true count" by dividing the running count

by an estimate of the number of decks left in the shoe.<sup>11</sup> For example, a running count of +10 with an estimated 2 decks remaining yields a true count of +5.

3. **Applying the True Count to Strategy:** The true count dictates two critical modifications to the agent's play:

- **Bet Sizing:** The fundamental principle of profitable card counting is to bet more when the advantage is in your favor. The AI will recommend a minimum bet when the true count is neutral or negative, and proportionally increase the bet size as the true count rises.<sup>9</sup>
- **Playing Deviations:** A high true count can alter the mathematically optimal play away from Basic Strategy. For instance, Basic Strategy dictates hitting a hard 16 against a dealer's 10. However, with a sufficiently high positive true count, the deck is so rich in 10s that the dealer is more likely to bust, making standing the more profitable play.<sup>11</sup> Similarly, taking the "insurance" bet, normally a poor choice, becomes profitable when the true count is +3 or higher because the likelihood of the dealer having a 10 in the hole is significantly increased.<sup>4</sup> These "deviations" are a critical component of a winning card counting strategy.

Human players face immense difficulty in perfectly executing a card counting strategy in a casino environment due to distractions, the need for concealment, and the mental strain of tracking multiple variables.<sup>11</sup> An AI agent, however, is immune to these limitations. It can flawlessly track the count, perform true count conversions with perfect deck estimation, and recall dozens of deviation indices without error. This allows the AI to not only implement a system like Hi-Lo perfectly but also to potentially use more complex and powerful Level 2 or 3 systems (e.g., Zen Count, which assigns different weights like -2 to 10s) that would be exponentially more difficult for a human to manage.<sup>8</sup> The AI's theoretical execution of card counting is perfect. The primary engineering challenges, therefore, are not in the game logic itself but in ensuring the vision system provides flawless input and in masking the agent's robotic precision to avoid detection by the online casino's security systems, which are trained to spot non-human betting patterns.<sup>14</sup>

| System Name  | Level | Card Tags (2-6 / 7-9 / 10-A) | Betting Correlation (BC) | Playing Efficiency (PE) | Balanced/ Unbalanced |
|--------------|-------|------------------------------|--------------------------|-------------------------|----------------------|
| Hi-Lo        | 1     | +1 / 0 / -1                  | 0.97                     | 0.51                    | Balanced             |
| K-O (Rookie) | 1     | +1 / 0 / -1                  | 0.98                     | 0.55                    | Unbalanced           |
| Red 7        | 1     | +1 (2-6),<br>+0.5 (Red       | 0.98                     | 0.54                    | Unbalanced           |

|             |   |                                                                     |      |      |          |
|-------------|---|---------------------------------------------------------------------|------|------|----------|
|             |   | 7) / 0 / -1                                                         |      |      |          |
| Zen Count   | 2 | +1 (2,3,7),<br>+2 (4,5,6) /<br>0 / -2 (10),<br>-1 (A)               | 0.96 | 0.77 | Balanced |
| Wong Halves | 3 | +0.5 (2,7),<br>+1 (3,4,6),<br>+1.5 (5) /<br>-0.5 (9) / -1<br>(10-A) | 1.00 | 0.70 | Balanced |

Note: BC, PE values are relative measures of effectiveness, with higher being better. Data synthesized from card counting system analyses.<sup>8</sup>

## Navigating Poker: A Game of Imperfect Information and Human Factors

In stark contrast to Blackjack, Poker is a game of imperfect information. The opponent's cards are hidden, making it impossible to calculate a single "correct" move. Optimal play requires moving beyond pure mathematics and into the realms of statistical inference, probability, and opponent psychology. The AI agent must be equipped with a suite of tools to quantify uncertainty, evaluate risk, and model the behavior of its human adversaries.

### Quantifying Hand Value: Equity Calculation and Hand Strength Algorithms

The first step in any poker decision is to understand the value of one's own hand. This value is not static; it is a probabilistic measure of its potential to win at showdown against the range of possible hands an opponent might hold. This concept is known as "equity".<sup>15</sup>

To begin, the agent requires a highly efficient **hand ranking algorithm**. Given seven cards (two hole cards and five community cards in Texas Hold'em), this algorithm must instantly determine the best possible five-card combination and assign it a unique rank (e.g., Royal Flush > Straight Flush > Four of a Kind, etc.).<sup>16</sup> For real-time performance, this is often

accomplished using bitwise operations or pre-computed lookup tables rather than brute-force enumeration of all 21 combinations.<sup>16</sup>

With a ranking system in place, the agent can calculate its equity via **Monte Carlo simulation**. Because the opponent's exact hand is unknown, we operate on a "range"—a collection of possible hands they might be playing in a given situation. The simulation process is as follows<sup>18</sup>.

1. Define a range of possible two-card hands for the opponent (e.g., {AA, KK, AKs}).
2. For each hand in that range, iterate thousands of times.
3. In each iteration, randomly deal the remaining community cards (the turn and/or river).
4. Use the hand ranking algorithm to determine the winner of the showdown.
5. Tally the wins, ties, and losses for our hand against the opponent's entire range. The win percentage (plus half the tie percentage) is our hand's equity.<sup>15</sup>

For a more nuanced evaluation, especially on early betting rounds like the flop, the AI can implement the **Effective Hand Strength (EHS)** algorithm.<sup>21</sup> EHS refines the concept of raw equity by explicitly accounting for the hand's future potential to improve or be outdrawn. The EHS is calculated with the formula:

$$EHS = HS \times (1 - NPOT) + (1 - HS) \times PPOT$$

Where:

- **HS** (Hand Strength) is the current raw equity against the opponent's range.
- **PPOT** (Positive Potential) is the probability that our hand, if currently behind, will improve to become the best hand on future streets.
- **NPOT** (Negative Potential) is the probability that our hand, if currently ahead, will be outdrawn and become a losing hand.

This calculation is computationally intensive as it requires simulating future cards for every possible opponent hand, but it provides a much more accurate assessment of a hand's true value, particularly for drawing hands.<sup>21</sup>

## The Mathematics of Betting: Pot Odds and Expected Value (EV)

Once the agent has an estimate of its equity, it can make fundamentally sound decisions about whether to call an opponent's bet. This is governed by the concept of **pot odds**, which compares the amount of money in the pot to the cost of the call.<sup>22</sup>

The AI will calculate the required equity to make a profitable call using a simple formula:

$$\text{Required Equity (\%)} = \frac{\text{Call Size}}{\text{Total Pot Size After Call}} = \frac{\text{Call}}{\text{Call} + \text{Pot}}$$

Size}{Current\ Pot\ +\ Bet\ Size\ +\ Call\ Size}\$\$

25

The decision rule is straightforward:

- If Hand Equity > Required Equity, the call has a positive expected value (+EV) and is profitable in the long run. The recommended action is to call.
- If Hand Equity < Required Equity, the call has a negative expected value (-EV) and will lose money in the long run. The recommended action is to fold.<sup>23</sup>

This comparison forms the mathematical backbone of poker strategy. For example, if the pot is \$150 and an opponent bets \$50, the total pot becomes \$200. To call, we must risk \$50 to win a total pot of \$250. The pot odds are 4:1 (\$200 to \$50). The required equity is  $\$50 / (\$200 + \$50) = 20\%$ . If our hand has more than a 20% chance of winning at showdown (i.e., its equity is >20%), the call is correct.

## The AI Frontier: Opponent Modeling

The calculations for equity and pot odds are only as reliable as the input they receive—specifically, the estimated range of the opponent's hand.<sup>28</sup> A player who folds most of their hands will have a much stronger range when they bet than a player who plays loosely. Therefore, the most critical and complex component of a poker AI is **opponent modeling**: the process of building a predictive model of an opponent's strategy based on their observed actions.<sup>29</sup>

The AI can employ several layers of opponent modeling, from simple to highly advanced:

- **Static Archetypes:** The simplest form involves classifying players based on key statistical tendencies gathered over many hands, such as VPIP (Voluntarily Put money In Pot) and PFR (Pre-Flop Raise percentage).<sup>32</sup> A player with low VPIP is "tight," while one with high VPIP is "loose." A player with a small gap between VPIP and PFR is "aggressive," while a large gap indicates a "passive" style. The AI can then assign a player to an archetype (e.g., Tight-Aggressive, Loose-Passive) and use a pre-defined set of hand ranges for that player type.<sup>33</sup>
- **Dynamic Weight Tables:** A more sophisticated, dynamic approach involves maintaining a probability distribution over all 1,326 possible starting hands for each opponent. This is known as a "weight table".<sup>30</sup> At the start of a hand, all hands might be considered equally likely (or weighted by a general starting hand chart). When an opponent takes an action—for example, raising from an early position—the AI updates these weights. The probabilities for strong hands like AA, KK, and AK are increased, while the probabilities for weak hands like 72o are decreased to near zero. This process of "re-weighting"

continues on every street, allowing the AI to progressively narrow and refine its estimate of the opponent's hand range based on their specific actions within the current hand.<sup>29</sup>

- **Advanced Machine Learning Models:** The state-of-the-art in opponent modeling utilizes deep learning. A **transformer-based model**, for instance, can be trained on millions of hand histories to analyze the sequence of actions taken by a player (bet sizes, timing, position, board texture) to predict their next action or infer their likely hand strength.<sup>34</sup> This moves beyond simple hand ranges to predictive behavioral analysis. The output of this transformer—a prediction of the opponent's action or a refined probability distribution of their holdings—can then be fed as an additional input into the agent's own decision-making model, allowing it to adapt its strategy in real-time to exploit the specific tendencies of that opponent.<sup>34</sup>

The relationship between opponent modeling and equity calculation is not sequential but symbiotic. The opponent model is not a separate, final step; it is a critical pre-processor for the equity calculator. An equity calculation performed against a generic or inaccurate range is mathematically meaningless. The power of a sophisticated poker AI comes from its ability to use the opponent model to generate a highly accurate, dynamically updated, weighted range of hands. This refined range then becomes the input for the Monte Carlo equity simulation. At each decision point, the opponent's actions provide new information, the model updates the range, and the equity is recalculated. This iterative loop of observation, inference, and calculation is the core of expert-level poker analysis.<sup>28</sup>

## The Unexploitable Ideal: Introduction to Game Theory Optimal (GTO) Play

While exploitative play based on opponent modeling seeks to maximize profit against specific, flawed opponents, **Game Theory Optimal (GTO)** play is a defensive strategy designed to be unexploitable.<sup>36</sup> A GTO strategy is an approximation of a Nash Equilibrium for the game. If two GTO players compete, they are expected to break even in the long run (minus the rake) because neither can exploit the other.<sup>36</sup>

GTO strategies are characterized by balance and mixed frequencies. For example, a GTO player will not always bet their strong hands and check their weak hands; instead, they will bet their strong hands *most* of the time and also bet a specific frequency of weak hands as bluffs. This makes it impossible for an opponent to know whether a bet represents strength or a bluff, making them indifferent to calling or folding. These strategies are calculated using complex algorithms like **Counterfactual Regret Minimization (CFR)**, where a program plays against itself for billions of hands to iteratively minimize its "regret" for not choosing other actions.<sup>38</sup>

For the proposed AI agent, a full real-time GTO solver is computationally infeasible for no-limit hold'em.<sup>36</sup> Instead, GTO serves as a powerful strategic baseline. The agent can be



programmed with pre-solved GTO solutions for common situations (e.g., pre-flop opening ranges from each position). The agent's default strategy will be to adhere to this GTO baseline. It will only deviate from this unexploitable foundation when its opponent modeling component identifies a clear, significant, and exploitable tendency in an opponent's play—for example, an opponent who folds to river bets 80% of the time. In this case, the agent can shift from a balanced GTO strategy to an exploitative one by bluffing the river far more frequently.<sup>40</sup> This hybrid approach combines the defensive security of GTO with the profit-maximizing aggression of exploitative play.

## Part 2: System Architecture and Technology Stack

This part translates the theoretical models into a concrete software blueprint. A modular, multi-layered architecture is designed to separate concerns—from visual input to logical processing to user output—ensuring the system is maintainable, scalable, and performant.

### Blueprint for the AI Agent: A Multi-Layered Architecture

The complexity of a real-time advisory system necessitates a layered architecture. Each layer has a distinct responsibility, communicating with adjacent layers through well-defined interfaces. This modularity allows for independent development, testing, and optimization of each component.

#### The Presentation Layer (The Overlay)

The Presentation Layer is the user-facing component, responsible for displaying the AI's analysis and recommendations. Its primary requirement is to function as an unobtrusive Heads-Up Display (HUD) that overlays the online game client. This means the window must be transparent, frameless, always-on-top, and, crucially, "click-through," allowing the user to interact with the game window underneath without interference.<sup>42</sup>

Several Python GUI frameworks can be considered for this task:

- **Tkinter:** As Python's standard GUI library, it is readily available. However, achieving advanced window effects like full transparency and click-through behavior is often

platform-dependent and can be complex to implement robustly.<sup>42</sup>

- **Kivy:** Designed for cross-platform and multi-touch applications, Kivy is powerful but may be overly complex for a desktop-only overlay. Its community and documentation are less focused on the low-level window manipulation required for this specific use case.<sup>46</sup>
- **PyQt/PySide:** These are Python bindings for the comprehensive C++ Qt framework. Qt provides extensive and granular control over window properties, making it the ideal choice. It offers direct access to window flags like `Qt.WindowStaysOnTopHint`, `Qt.FramelessWindowHint`, and attributes like `Qt.WA_TranslucentBackground`, which are essential for creating a proper overlay.<sup>48</sup> Despite a steeper learning curve, **PyQt is the recommended framework** for its power, maturity, and control over native windowing systems.<sup>51</sup>

The UI design must prioritize clarity and information density. It should present critical data—such as recommended actions ("HIT"), win probabilities, pot odds, and opponent statistics—in a clean, minimalist fashion that does not obscure the underlying game table. Design cues can be taken from established poker tracking software and blackjack training applications.<sup>53</sup>

## The Vision Layer (The "Eyes")

This layer is the system's sensory input, responsible for capturing the screen and translating visual information into structured data for the Logic Layer. This is the most critical component for real-time performance and overall system accuracy.

The vision pipeline consists of three main stages:

1. **Screen Capture:** Continuous, high-framerate screen capture is paramount. Standard Python libraries like `pyautogui` and `PIL.ImageGrab` are generally too slow for this task, often achieving only a few frames per second, which is inadequate for a fast-paced card game.<sup>57</sup> High-performance libraries designed for low-latency capture are required. **mss** and **vidgear's ScreenGear** are excellent choices, as they can capture screen regions at high FPS directly into NumPy arrays, the standard format for image processing with OpenCV, thus avoiding slow disk I/O or format conversions.<sup>41</sup>
2. **Object Detection:** To locate and identify game elements like playing cards, suits, ranks, and poker chips, a Convolutional Neural Network (CNN) is the most robust solution. A **YOLO (You Only Look Once)** model, such as YOLOv8, is particularly well-suited due to its exceptional balance of speed and accuracy, making it ideal for real-time detection.<sup>60</sup> This model must be custom-trained on a large dataset of annotated screenshots from the specific online poker or blackjack clients the agent will be used with. This ensures the model is robust to the specific art style, lighting, and layout of the target application.<sup>60</sup>

The model will output bounding boxes and class labels for each detected object (e.g., "Ace of Spades," "10 of Hearts").

3. **Optical Character Recognition (OCR):** To read text-based information such as player names, stack sizes, bet amounts, and the total pot, an OCR engine is necessary. While pytesseract is a common wrapper for Google's Tesseract engine, it can be inefficient for real-time applications as it may re-initialize the engine on each call.<sup>63</sup> More suitable alternatives include **tesseractocr**, which provides direct Python bindings to the Tesseract C++ API for better performance, or **EasyOCR**, which is GPU-accelerated and often performs better on stylized text found in game interfaces.<sup>64</sup> For optimal accuracy, the image regions containing text, identified by the object detection model or static coordinates, should be pre-processed using OpenCV functions like grayscaling, thresholding, and normalization before being passed to the OCR engine.<sup>63</sup>

The performance of the entire AI agent is fundamentally constrained by the speed and accuracy of this vision pipeline. A lag in screen capture means the AI's advice is based on an outdated game state. A single misidentified card by the YOLO model can invalidate all subsequent strategic calculations, leading to catastrophically incorrect recommendations. An error in reading a bet size by the OCR module will result in a flawed pot odds calculation. These components are not just steps in a process; they are a fragile chain where any weak link compromises the integrity of the entire system. Consequently, a significant portion of the development and testing effort must be dedicated to creating a fast, accurate, and robust Vision Layer that can withstand minor UI updates from the game provider.

## The Logic Layer (The "Brain")

This layer serves as the central processing unit of the agent. It receives structured data from the Vision Layer (e.g., {'player\_hand': ['As', 'Kh'], 'pot\_size': 150}), maintains a comprehensive model of the current game state, queries the appropriate decision engine, and passes the results to the Presentation Layer for display.

Its core components include:

- **Game State Machine:** This component tracks the progression of the game. For poker, it would manage states like PREFLOP, FLOP, TURN, and RIVER, keeping track of the active player, pot size, community cards, and previous actions.<sup>16</sup> For blackjack, it would track the player's turn, dealer's turn, and the status of each split hand.
- **Decision Engines:** This module contains the implementation of the game theory models from Part 1. It will be organized into sub-modules:
  - **Blackjack Engine:** Encapsulates the logic for Basic Strategy, the chosen card counting system (e.g., Hi-Lo), true count calculation, and playing deviations.

- **Poker Engine:** A more complex module containing the Hand Evaluator, the Monte Carlo Equity Calculator, the Pot Odds Calculator, and the Opponent Modeler.
- **Main Controller:** This orchestrates the entire process. It operates in a continuous loop: it receives an updated game state from the Vision Layer, identifies the current decision point (e.g., it is our turn to act), calls the relevant decision engine with the current state, receives the calculated odds and recommended action, formats this information, and pushes it to the UI for display.

## The Data Layer

This layer is responsible for data persistence, which is primarily required for the poker agent's opponent modeling capabilities.

- **Database:** A lightweight, serverless database is sufficient for this purpose. **SQLite**, which is included in Python's standard library, is an excellent choice. It is file-based, requires no separate server process, and is more than capable of handling the storage of hand histories and player statistics.<sup>64</sup>
- **Data Models:** Clear database schemas will be defined to store relevant information. This would include tables for players (storing long-term stats like VPIP, PFR), sessions (grouping hands played in a single session), and hand\_histories (recording the cards, positions, and sequence of actions for every player in a hand). This historical data is then used to train and refine the opponent models over time.

## Project Structure and Language Selection

A well-organized project is crucial for managing complexity and facilitating future development.

- **Language: Python** is the definitive choice for this project. Its extensive ecosystem of mature libraries for machine learning (PyTorch, TensorFlow), computer vision (OpenCV-Python), scientific computing (NumPy), GUI development (PyQt6), and high-performance screen capture (mss) makes it uniquely suited to integrate all the disparate components of this system into a cohesive whole.<sup>59</sup>
- **Project Directory Structure:** A modular directory structure will be employed to enforce separation of concerns:  
[/card-ai-agent](#)

```

|
|-- config/ # Configuration files (e.g., blackjack_rules.json, ui_settings.yaml)
|-- data/ # Persistent data (e.g., poker_database.db, yolo_model.pt, training_images/)
|-- docs/ # Project documentation (e.g., SRS.md)
|-- src/ # Main source code
| |-- main.py # Application entry point
| |-- vision/ # Screen capture, object detection, and OCR modules
| | |-- screen_capture.py
| | |-- object_detector.py
| | -- ocr.py | |-- logic/ # Game logic and decision-making engines | | |-- game_state.py | | |--
blackjack_engine.py | | -- poker_engine/
| | |-- hand_evaluator.py
| | |-- equity_calculator.py
| | -- opponent_modeler.py | |-- ui/ # PyQt6-based overlay, widgets, and styling | | |--
overlay_window.py | | -- widgets.py
| -- utils/ # Shared utilities and constants (e.g., card_definitions.py) | -- tests/ # Unit and
integration tests for all modules | -- requirements.txt # List of Python package dependencies
-- README.md # Project overview and setup instructions
...

```

| Architectural Layer | Purpose                               | Recommended Library/Technology |
|---------------------|---------------------------------------|--------------------------------|
| Presentation        | Transparent, click-through UI overlay | PyQt6 or PySide6               |
| Vision              | Real-time screen capture              | mss or vidgear.ScreenGear      |
|                     | Object detection (cards, chips)       | PyTorch / YOLOv8               |
|                     | Text recognition (pot size, bets)     | EasyOCR or tesseract           |
|                     | Image processing                      | OpenCV-Python, NumPy           |
| Logic               | Blackjack decision engine             | Custom Python implementation   |
|                     | Poker decision engine                 | Custom Python                  |

|      |                                      |                                   |
|------|--------------------------------------|-----------------------------------|
|      |                                      | implementation                    |
|      | Poker hand evaluation                | treys or custom bitwise evaluator |
| Data | Hand history & opponent data storage | SQLite (via sqlite3 module)       |

## Part 3: Implementation Guide and AI-Assisted Coding

This section provides actionable steps and specific, well-formulated prompts designed for use with advanced AI code generation models. These prompts translate the architectural components defined in Part 2 into functional Python code, accelerating the development process.

### Building the Vision Pipeline

The Vision Pipeline is the sensory system of the agent. Its implementation requires a focus on performance and accuracy.

#### High-FPS Screen Capture

To ensure the agent receives game state information in real-time, a high-performance, non-blocking screen capture module is essential.

- Prompt for AI Code Generator:**  
As an expert Python developer, write a complete, self-contained class named `ScreenCapturer` that uses the `mss` and `opencv-python` libraries.

The class must meet these requirements:

- The constructor `__init__(self, bounding_box)` should accept a dictionary `bounding_box` with keys 'top', 'left', 'width', and 'height'.

2. It must have a ``start()`` method that begins capturing frames in a separate daemon thread to avoid blocking the main application loop.
3. The capture loop within the thread should continuously grab screenshots of the specified bounding box, convert them into a BGR NumPy array suitable for OpenCV, and store the latest frame in an instance variable.
4. Implement a ``get_latest_frame()`` method that returns the most recent captured frame. This method should be thread-safe.
5. Implement a ``stop()`` method that gracefully terminates the capture thread.
6. Include type hinting and docstrings for all methods.

This prompt is based on best practices for real-time applications, leveraging threading for non-blocking I/O and using efficient libraries like mss.<sup>58</sup>

## Training a Custom YOLO Model

The agent's ability to "see" the cards depends on a custom-trained object detection model.

- **Data Collection and Annotation:** Before coding, a dataset must be created. This involves taking several hundred screenshots of the target online game client, ensuring a wide variety of game states (different cards, stack sizes, lighting, etc.). Using an annotation tool like Roboflow or CVAT, one must then draw bounding boxes around every object of interest (e.g., each of the 52 cards, different colored chips) and assign a class label. The annotated data should be exported in YOLO format.
- **Prompt for AI Code Generator:**  
Generate a complete Python script to train a YOLOv8 object detection model for a custom dataset using the ``ultralytics`` library.

The script should perform the following steps:

1. Import the ``YOLO`` class from ``ultralytics``.
2. Define the path to the dataset's YAML configuration file. This YAML file specifies the paths to training/validation images and the class names, like so:

```
train: /yolov5/dataset/train/images
val: /yolov5/dataset/valid/images
nc: 52
names: ['2c', '2d', ..., 'As']
```
3. Load a pre-trained YOLOv8 model (e.g., `'yolov8n.pt'`) to leverage transfer learning.
4. Call the ``train()`` method on the model instance, passing arguments for the data configuration file, number of epochs (e.g., 100), image size (e.g., 640), and batch size.
5. The script should automatically save the best performing model weights (``best.pt``) in a `'runs/detect/train'` directory.

6. Include comments explaining each step of the process.

This prompt follows standard procedures for training custom YOLO models, emphasizing the use of pre-trained weights and a structured dataset configuration.<sup>60</sup>

## Implementing OCR for Game Text

To read dynamic text like bet sizes and pot totals, a reliable OCR module is needed.

- **Prompt for AI Code Generator:**

Write a Python function `extract_text_from_image(image_crop: np.ndarray) -> str`.

This function should:

1. Accept a NumPy array representing a cropped portion of the screen.
2. Use `'opencv-python'` to pre-process the image for better OCR accuracy. The pre-processing steps should include:
  - a. Converting the image to grayscale.
  - b. Applying an adaptive threshold (`'cv2.ADAPTIVE_THRESH_GAUSSIAN_C'`) to create a binary image, which helps with varying lighting conditions.
  - c. Optionally, applying a slight Gaussian blur to reduce noise.
3. Use the `'easyocr'` library to perform OCR on the pre-processed image. Initialize the `'easyocr.Reader'` for the English language and with GPU support enabled if available.
4. The function should parse the results from EasyOCR, concatenate any detected text fragments, and return a single, cleaned string containing only digits and decimal points.
5. Handle cases where no text is detected by returning an empty string.

This prompt specifies not only the tool (EasyOCR) but also crucial pre-processing steps that are vital for achieving high accuracy on non-document images.<sup>63</sup>

## Implementing the Game Logic Engines

These modules form the "brain" of the agent, applying the strategies from Part 1 to the data from the Vision Pipeline.

### Blackjack Basic Strategy Engine



This module will serve as a lookup table for the mathematically optimal move.

- **Prompt for AI Code Generator:**

Create a Python class named `BlackjackEngine`.

The class should have the following features:

1. The constructor `__init__(self, rules_filepath)` loads a specific Basic Strategy chart from a JSON file. The JSON should be structured as a nested dictionary: `{ "h17": { "hard": { "16": { "10": "H", ... } }, "soft": { ... }, "pairs": { ... } } }`.
2. A method `get_recommendation(self, player_hand: list, dealer_up_card: int, game_rules: dict) -> str` that determines the correct move.
3. The `player_hand` is a list of card strings (e.g., `['A', '6']`). The `dealer_up_card` is an integer value. The `game_rules` dictionary specifies variations like `{ 'dealer_hits_soft_17': True, 'double_after_split': True }`.
4. The method should first determine if the hand is a pair, a soft total, or a hard total.
5. It should then look up the correct move from the loaded strategy chart based on the hand type, player total, dealer card, and game rules.
6. It must return the move as a string: 'H' (Hit), 'S' (Stand), 'D' (Double), 'P' (Split), or 'R' (Surrender).

This prompt structures the logic clearly, separating the strategy data (JSON) from the decision-making code and accounting for rule variations.<sup>1</sup>

## Poker Equity Calculator

This module performs the computationally intensive Monte Carlo simulations.

- **Prompt for AI Code Generator:**

Write a Python function `calculate_poker_equity` that calculates hand equity in Texas Hold'em using a Monte Carlo simulation.

The function signature should be: `calculate_equity(my_hand: list, opponent_range: list, board: list =, num_simulations: int = 10000) -> dict`.

Requirements:

1. It must use the `treys` library for hand evaluation.
2. `my_hand`, `opponent_range`, and `board` are lists of card strings (e.g., `my_hand=['As', 'Kh']`). `opponent_range` is a list of possible two-card hands for the opponent.

3. The function should loop for ``num_simulations`` iterations.
4. In each iteration:
  - a. Randomly select one hand for the opponent from ``opponent_range``.
  - b. Create a deck of the remaining cards (52 minus ``my_hand``, the selected opponent hand, and the ``board``).
  - c. Randomly draw the remaining community cards needed to complete a 5-card board.
  - d. Use ``treys.Evaluator`` to determine the rank of ``my_hand`` and the opponent's hand.
  - e. Tally the outcome (win, tie, or loss) for ``my_hand``.
5. Return a dictionary with the final percentages: ``{'win': 55.5, 'tie': 2.1, 'loss': 42.4}``.

This prompt specifies a well-known and efficient library (treys) and clearly outlines the logic of a Monte Carlo simulation, which is the standard approach for this problem.<sup>19</sup>

## Constructing the User Interface

The UI must be functional and unobtrusive, serving as a clean data-display layer.

### Transparent Overlay Window

This is the foundational element of the user interface.

- **Prompt for AI Code Generator:**

Using the PyQt6 library in Python, create a class ``OverlayWindow`` that inherits from ``QMainWindow``.

This class must create a window with the following properties:

1. It must be frameless (no title bar or borders).
2. It must always stay on top of all other windows.
3. The window background must be fully transparent.
4. The window must be "click-through," meaning mouse clicks should pass through it to the window underneath. This requires setting the ``Qt.WindowType.WindowTransparentForInput`` flag.
5. It should be maximized to fill the entire screen on startup.
6. Include a basic ``main`` block to instantiate and show the window.

This prompt is highly specific about the required PyQt window flags and attributes

needed to create a functional, non-interactive overlay, which is a non-trivial task.<sup>48</sup>

## Displaying Data on the Overlay

Once the overlay exists, it needs to be populated with data from the Logic Layer.

- **Prompt for AI Code Generator:**

Extend the `OverlayWindow` class from the previous prompt. Add a public method `update\_display(self, display\_elements: list)`.

The `display\_elements` parameter will be a list of dictionaries, where each dictionary represents a piece of text to display, e.g., `{ 'text': 'HIT', 'pos': (x, y), 'color': 'green' }`.

The `update\_display` method should:

1. First, clear any existing text labels from the previous update to prevent artifacts.
2. Iterate through the `display\_elements` list.
3. For each element, create a new `QLabel` widget.
4. Set the label's text, position it at the specified `(x, y)` coordinates, and style it using a stylesheet to have the specified text color, a semi-transparent dark background for readability, and a bold font.
5. Ensure the new labels are shown.

## Part 4: Project Execution and Ethical Considerations

This final part addresses the practicalities of project management and, most importantly, the critical legal and ethical constraints that define the project's legitimate use. A technically brilliant tool is useless if its application is prohibited.

### Formalizing the Project: The Software Requirements Specification (SRS)

A Software Requirements Specification (SRS) is a formal document that serves as the

blueprint for a software project. It provides a complete description of the system's purpose, features, functionality, and performance requirements. Creating a detailed SRS is a crucial first step that aligns developers and stakeholders, reduces ambiguity, and provides a stable baseline for development and testing.<sup>70</sup>

For a complex AI-driven project like this, the SRS is vital for defining precise performance and accuracy targets for the machine learning components. The following is a template for an SRS.md file, structured according to industry best practices.<sup>73</sup>

---

**SRS.md**

# **Software Requirements Specification for Cardsharp AI Assistant**

**Version 1.0**

## **1. Introduction**

### **1.1 Purpose**

This document specifies the requirements for the Cardsharp AI Assistant, a real-time software tool designed for educational and research purposes. The system overlays an online card game client to provide statistical analysis and strategic recommendations for Blackjack and Texas Hold'em poker. Its primary goal is to serve as a training aid for players to learn optimal strategies.

### **1.2 Scope**

The system will:

- Analyze on-screen gameplay of specific, pre-configured online Blackjack and Poker clients.
- Provide real-time recommendations for Blackjack based on Basic Strategy and a Hi-Lo card count.
- Provide real-time analysis for Poker, including hand equity, pot odds, and basic opponent statistics.
- Display all information via a non-interactive, transparent screen overlay.

The system will **not**:

- Automate any user actions (e.g., clicking buttons, placing bets).
- Be used for real-money gambling.
- Interface directly with any game client's software or network traffic.

## 1.3 Definitions, Acronyms, and Abbreviations

- **AI**: Artificial Intelligence
- **HUD**: Heads-Up Display
- **OCR**: Optical Character Recognition
- **RTA**: Real-Time Assistance
- **GTO**: Game Theory Optimal
- **SRS**: Software Requirements Specification

# 2. Overall Description

## 2.1 Product Perspective

The Cardsharp AI Assistant is a standalone desktop application that operates by visually interpreting the user's screen. It is an independent, advisory tool with no direct integration into any third-party gaming software.

## 2.2 Product Features

- **Blackjack Module:** Provides recommendations for Hit, Stand, Double, Split, and Surrender. Displays the current running count and true count.
- **Poker Module:** Displays player hole cards, community cards, pot odds, and hand equity against a configurable number of opponents. Tracks and displays basic opponent VPIP/PFR statistics for the current session.
- **Vision System:** Captures the screen, detects cards and chips, and reads text values.
- **Overlay UI:** A transparent, click-through window to display all generated information.

## 2.3 User Characteristics

The intended user is a student of card games or an AI enthusiast with a technical understanding of game theory and probability.

## 2.4 Constraints

- The system must run on modern Windows and macOS operating systems.
- The system must be developed in Python 3.10+.
- The end-to-end latency must be low enough to be useful in a live-paced game.

# 3. System Features and Functional Requirements

## 3.1 Vision System

- **REQ-V1:** The system shall capture a user-defined screen region at a minimum sustained rate of 10 frames per second.
- **REQ-V2:** The system's object detection model shall identify all 52 standard playing cards

from the target game client UI with a classification accuracy of greater than 99.5% under typical operating conditions.

- **REQ-V3:** The system shall correctly identify the location of player hole cards, community cards, and dealer cards with >99% accuracy.
- **REQ-O1:** The system shall read numerical text from the screen (e.g., pot size, bet amounts, stack sizes) with a character-level accuracy of greater than 98%.

## 3.2 Blackjack Module

- **REQ-B1:** The system shall provide correct Blackjack Basic Strategy recommendations for a configurable set of table rules (H17/S17, DAS, LS, Number of Decks).
- **REQ-B2:** The system shall maintain an accurate running count and true count based on all visible cards dealt from the shoe.
- **REQ-B3:** The system shall provide correct playing deviation recommendations based on the current true count.

## 3.3 Poker Module

- **REQ-P1:** The system shall calculate poker hand equity via Monte Carlo simulation against a specified number of random opponent hands with a margin of error of +/- 2% over 10,000 simulations.
- **REQ-P2:** The system shall accurately calculate and display the pot odds required for a player to call a bet.
- **REQ-P3:** The system shall track and display session-specific VPIP and PFR statistics for each opponent at the table.

# 4. External Interface Requirements

## 4.1 User Interfaces

The system shall present a single graphical user interface in the form of a full-screen,

transparent, click-through overlay. All information shall be displayed as text elements with semi-transparent backgrounds for readability.

## 4.2 Software Interfaces

The system will interface with the operating system's window manager to create the overlay and with screen capture APIs to acquire visual data. It will have no other software interfaces.

# 5. Non-Functional Requirements

## 5.1 Performance

- **PERF-1:** The end-to-end latency, from a change occurring on screen (e.g., a card being dealt) to the corresponding update appearing on the UI overlay, shall be less than 750 milliseconds on average.

## 5.2 Usability

- **USE-1:** The overlay must not obstruct critical game information such as player action buttons or bet sliders.
- **USE-2:** The user must be able to toggle the visibility of the overlay with a single hotkey.

## 5.3 Reliability

- **REL-1:** The application must handle unexpected game state changes (e.g., players leaving, new game starting) without crashing. It should reset its state and continue analysis.



---

## A Critical Warning: Navigating Terms of Service and Legality

While the creation of this AI agent is a formidable and educational technical challenge, its application in real-money online environments is unequivocally prohibited and constitutes cheating. All major online gaming platforms have explicit and stringent rules against the use of such tools, and they invest heavily in detection and enforcement.

An analysis of the Terms of Service from leading online poker sites reveals a zero-tolerance policy:

- **PokerStars:** Their policy explicitly prohibits any tool or service that "plays without human intervention ('a bot') or reduces the requirement of a human to make decisions" and any tool that "offers real-time advice on what action to take".<sup>77</sup> Their extensive list of banned software includes many applications far less sophisticated than the agent proposed here. Furthermore, their client software is known to scan for prohibited running processes on a user's computer.<sup>14</sup>
- **888poker:** The user agreement forbids any "software program, robot or external aid, which is endowed with artificial intelligence".<sup>79</sup> The platform has a documented history of banning accounts and confiscating player funds for the suspected use of AI-based tools, demonstrating their commitment to enforcing this rule.<sup>79</sup>
- **GGPoker:** This platform enforces a strict policy against **Real-Time Assistance (RTA)**, which is broadly defined as any external assistance that provides an unfair advantage or influences decision-making in real-time. This includes software, apps, websites, and even physical reference materials like charts.<sup>81</sup> Violation of this policy can lead to a permanent ban and seizure of all funds.<sup>84</sup>
- **Partypoker:** Their policy bans "assistance software" and "strategic material," which includes any external material that provides guidance or recommendations while playing.<sup>85</sup> They also prohibit the use of virtual machines and remote access software, indicating an awareness of methods players might use to circumvent detection.<sup>85</sup>

The sophistication of detection methods employed by these platforms cannot be overstated. They utilize dedicated game integrity teams that employ a combination of automated systems, statistical analysis of gameplay data, and machine learning to identify patterns indicative of non-human play.<sup>80</sup> An AI agent, even one programmed with random delays, is likely to exhibit subtle but statistically significant patterns in its decision timing, bet sizing, and strategic consistency that differ from a human player's profile. These are precisely the anomalies that security systems are designed to flag.<sup>14</sup>

The consequences of detection are severe and consistently enforced across the industry: immediate and permanent suspension of the player's account and the confiscation of the

entire account balance.<sup>14</sup> As the use of such software is a direct violation of the user agreement—a legally binding contract accepted by the player upon registration—there is no legal recourse for the recovery of these funds.<sup>89</sup>

Therefore, this project must be approached with the unequivocal understanding that it is **not a viable tool for gaining an advantage in real-money online gambling**. Its development and use must be strictly limited to legitimate and ethical applications, such as:

- **Academic Research:** Studying game theory, AI decision-making, and computer vision.
- **Personal Education:** Using the tool against play-money opponents or game simulators to learn and practice optimal strategies.
- **Post-Session Analysis:** Using the tool to analyze one's own hand histories after a game session has concluded to identify leaks and improve future play. This is a practice permitted by many sites.

Any attempt to use this agent in a live, real-money environment is a high-risk endeavor that will almost certainly result in financial loss and a permanent ban from the online gaming community. This legal and ethical boundary is the single most important constraint governing this project.

## Works cited

1. Blackjack Chart Strategies - Master the Odds This October - ReadWrite, accessed October 16, 2025, <https://readwrite.com/gambling/guides/blackjack-chart/>
2. How a Blackjack Basic Strategy Chart Saved My Precious Money - Jim Makos, accessed October 16, 2025, <https://jimmakos.com/blackjack-basic-strategy-chart/>
3. Blackjack Basic Strategy Chart : r/vegas - Reddit, accessed October 16, 2025, [https://www.reddit.com/r/vegas/comments/1b032o5/blackjack\\_basic\\_strategy\\_chart/](https://www.reddit.com/r/vegas/comments/1b032o5/blackjack_basic_strategy_chart/)
4. Blackjack Strategy Charts - How to Play Perfect Blackjack, accessed October 16, 2025, <https://www.blackjackapprenticeship.com/blackjack-strategy-charts/>
5. Basic Blackjack Strategy Charts | William Hill, accessed October 16, 2025, <https://news.williamhill.com/casino-guides/basic-blackjack-strategy-with-charts/>
6. How to Memorize Basic Strategy - Blackjack Apprenticeship, accessed October 16, 2025, <https://www.blackjackapprenticeship.com/how-to-memorize-basic-strategy/>
7. Blackjack - Wikipedia, accessed October 16, 2025, <https://en.wikipedia.org/wiki/Blackjack>
8. Card counting - Wikipedia, accessed October 16, 2025, [https://en.wikipedia.org/wiki/Card\\_counting](https://en.wikipedia.org/wiki/Card_counting)
9. Card Counting Systems - Blackjack Apprenticeship, accessed October 16, 2025, <https://www.blackjackapprenticeship.com/card-counting-systems/>
10. How To Count Cards in Blackjack and Bring Down the House, accessed October

- 16, 2025, <https://www.blackjackapprenticeship.com/how-to-count-cards/>
11. Card Counting in Blackjack - Introduction & Basics - Wizard of Odds, accessed October 16, 2025, <https://wizardofodds.com/games/blackjack/card-counting/introduction/>
  12. ELI5: How does counting cards work in blackjack? : r/explainlikeimfive - Reddit, accessed October 16, 2025, [https://www.reddit.com/r/explainlikeimfive/comments/1rue3n/eli5\\_how\\_does\\_counting\\_cards\\_work\\_in\\_blackjack/](https://www.reddit.com/r/explainlikeimfive/comments/1rue3n/eli5_how_does_counting_cards_work_in_blackjack/)
  13. How To Count Cards: Take Back the Edge at Blackjack - YouTube, accessed October 16, 2025, <https://www.youtube.com/watch?v=WkGldDHLb5k>
  14. Does PokerStars Have Bots? Here are the Facts [2025] | BlackRain79 - Elite Poker Strategy, accessed October 16, 2025, <https://www.blackrain79.com/2019/06/pokerstars-bots.html>
  15. Estimating the strength of poker hands by integer linear programming techniques, accessed October 16, 2025, [https://www.researchgate.net/publication/280085229\\_Estimating\\_the\\_strength\\_of\\_poker\\_hands\\_by\\_integer\\_linear\\_programming\\_techniques](https://www.researchgate.net/publication/280085229_Estimating_the_strength_of_poker_hands_by_integer_linear_programming_techniques)
  16. Essential Algorithms for Poker Hand Ranking & Game Logic - Innosoft Group, accessed October 16, 2025, <https://innosoft-group.com/essential-algorithms-for-poker-hand-ranking-game-logic/>
  17. The simplest algorithm for poker hand evaluation - Stack Overflow, accessed October 16, 2025, <https://stackoverflow.com/questions/10363927/the-simplest-algorithm-for-poker-hand-evaluation>
  18. johnsoong216/PokerOddsCalc: A simple tool to calculate poker hand odds (Omaha/Hold'em) - GitHub, accessed October 16, 2025, <https://github.com/johnsoong216/PokerOddsCalc>
  19. rendely/poker - GitHub, accessed October 16, 2025, <https://github.com/rendely/poker>
  20. ktseng/holdem\_calc: Texas Hold'em Odds Calculator - GitHub, accessed October 16, 2025, [https://github.com/ktseng/holdem\\_calc](https://github.com/ktseng/holdem_calc)
  21. Effective hand strength algorithm - Wikipedia, accessed October 16, 2025, [https://en.wikipedia.org/wiki/Effective\\_hand\\_strength\\_algorithm](https://en.wikipedia.org/wiki/Effective_hand_strength_algorithm)
  22. Pot odds - Wikipedia, accessed October 16, 2025, [https://en.wikipedia.org/wiki/Pot\\_odds](https://en.wikipedia.org/wiki/Pot_odds)
  23. Introduction to Pot Odds | How to Calculate Pot Odds in Poker, accessed October 16, 2025, <https://upswingpoker.com/pot-odds-step-by-step/>
  24. How to Calculate Poker Pot Odds, accessed October 16, 2025, <https://www.consciouspoker.com/blog/how-to-calculate-poker-pot-odds/>
  25. Calculating Pot Odds in Poker - What Is the Best Way? - PokerCoaching.com, accessed October 16, 2025, <https://pokercoaching.com/blog/pot-odds-in-poker/>
  26. Pot Odds Calculator - MicroGrinder Poker School, accessed October 16, 2025, <https://microgrinder.com/poker-calculators/pot-odds-calculator/>
  27. Pot odds calculations at the table : r/poker - Reddit, accessed October 16, 2025,

- [https://www.reddit.com/r/poker/comments/17px83a/pot\\_odds\\_calculations\\_at\\_the\\_table/](https://www.reddit.com/r/poker/comments/17px83a/pot_odds_calculations_at_the_table/)
28. 5.2 Hand Strength, accessed October 16, 2025, <https://webdocs.cs.ualberta.ca/~jonathan/PREVIOUS/Grad/papp/node38.html>
  29. Opponent Modeling in Texas Hold'em - Utrecht University Student Theses Repository Home, accessed October 16, 2025, <https://studenttheses.uu.nl/bitstream/handle/20.500.12932/8062/OpponentModelinginTexasHoldem.pdf?sequence=1&isAllowed=y>
  30. Improved Opponent Modeling in Poker - Computer Poker Research ..., accessed October 16, 2025, <https://poker.cs.ualberta.ca/publications/ICA100.pdf>
  31. Opponent Modeling in Poker, accessed October 16, 2025, <https://poker.cs.ualberta.ca/publications/AAAI98.pdf>
  32. Adapting Strategies to Opponent Models in Incomplete Information Games: A Reinforcement Learning Approach for Poker - ResearchGate, accessed October 16, 2025, [https://www.researchgate.net/publication/262204841\\_Adapting\\_Strategies\\_to\\_Opponent\\_Models\\_in\\_Incomplete\\_Information\\_Games\\_A\\_Reinforcement\\_Learning\\_Approach\\_for\\_Poker](https://www.researchgate.net/publication/262204841_Adapting_Strategies_to_Opponent_Models_in_Incomplete_Information_Games_A_Reinforcement_Learning_Approach_for_Poker)
  33. What Reinforcement Learning Method Should I Use for Poker AI with LLMs? - Reddit, accessed October 16, 2025, [https://www.reddit.com/r/LocalLLaMA/comments/1j2q5ym/what\\_reinforcement\\_learning\\_method\\_should\\_i\\_use/](https://www.reddit.com/r/LocalLLaMA/comments/1j2q5ym/what_reinforcement_learning_method_should_i_use/)
  34. Extended Abstract - CS 224R Deep Reinforcement Learning, accessed October 16, 2025, <https://cs224r.stanford.edu/projects/pdfs/cs224rFinalProject.pdf>
  35. Opponent Modeling in Deep Reinforcement Learning, accessed October 16, 2025, <http://proceedings.mlr.press/v48/he16.pdf>
  36. A Survey on Game Theory Optimal Poker - arXiv, accessed October 16, 2025, <https://arxiv.org/html/2401.06168v1>
  37. Game Theory Optimal (GTO) Texas Holdem Poker Theory - Cornell blogs, accessed October 16, 2025, <https://blogs.cornell.edu/info2040/2021/11/03/game-theory-optimal-gto-texas-holdem-poker-theory/>
  38. Poker AI - Optimal Strategy on the App Store - Apple, accessed October 16, 2025, <https://apps.apple.com/us/app/poker-ai-optimal-strategy/id1553727514>
  39. AI and the Future of Poker | GTO Wizard, accessed October 16, 2025, <https://blog.gtowizard.com/ai-and-the-future-of-poker/>
  40. Learning Strategies for Opponent Modeling in Poker - The Association for the Advancement of Artificial Intelligence, accessed October 16, 2025, <https://cdn.aaai.org/ocs/ws/ws1059/7132-30507-1-PB.pdf>
  41. Efficient Opponent Exploitation in No-Limit Texas Hold'em Poker: A ..., accessed October 16, 2025, <https://www.mdpi.com/2079-9292/10/17/2087>
  42. How do I create a transparent window canvas but keep the image on it visible? - Reddit, accessed October 16, 2025, [https://www.reddit.com/r/learnpython/comments/150uyog/how\\_do\\_i\\_create\\_a\\_transparent\\_window\\_canvas\\_but/](https://www.reddit.com/r/learnpython/comments/150uyog/how_do_i_create_a_transparent_window_canvas_but/)

43. jfd02/tkinter-transparent-window: Transparent & click through tkinter window. - GitHub, accessed October 16, 2025, <https://github.com/jfd02/tkinter-transparent-window>
44. PySimpleGUI how to make transparent click-through window? - Stack Overflow, accessed October 16, 2025, <https://stackoverflow.com/questions/60213167/pysimplegui-how-to-make-transparent-click-through-window>
45. Transparent window in Tkinter - GeeksforGeeks, accessed October 16, 2025, <https://www.geeksforgeeks.org/python/transparent-window-in-tkinter/>
46. Python GUI : Difference between tkinter, PyQt, and Kivy | by Qasim Uddin - Medium, accessed October 16, 2025, <https://medium.com/@qasim.coder/python-gui-smackdown-unleashing-the-power-of-tkinter-pyqt-and-kivy-e7b05d0e862>
47. Tkinter or Kivy or....? - Python Forum, accessed October 16, 2025, <https://python-forum.io/thread-8165.html>
48. PyQt5 - How to create semi transparent window ? - GeeksforGeeks, accessed October 16, 2025, <https://www.geeksforgeeks.org/python/pyqt5-how-to-create-semi-transparent-window/>
49. python - Transparent window with blur behind with pyqt - Stack Overflow, accessed October 16, 2025, <https://stackoverflow.com/questions/54807743/transparent-window-with-blur-behind-with-pyqt>
50. Creating a transparent overlay with qt - python - Stack Overflow, accessed October 16, 2025, <https://stackoverflow.com/questions/25950049/creating-a-transparent-overlay-with-qt>
51. Which GUI module is better in Python? tkinter or PyQt or kivy? - Reddit, accessed October 16, 2025, [https://www.reddit.com/r/Python/comments/123b6x2/which\\_gui\\_module\\_is\\_better\\_in\\_python\\_tkinter\\_or/](https://www.reddit.com/r/Python/comments/123b6x2/which_gui_module_is_better_in_python_tkinter_or/)
52. Comparing Python GUI Libraries: PyQt, Kivy, Tkinter, PySimpleGUI, WxPython, and PySide, accessed October 16, 2025, <https://blog.stackademic.com/comparing-python-gui-libraries-pyqt-kivy-tkinter-pysimplegui-wxpython-and-pyside-805f44039d8f>
53. Poker Copilot: Poker HUD Software, accessed October 16, 2025, <https://pokercopilot.com/>
54. How Poker Players Use Tech Tools to Analyze, Improve, and Win in 2025 - Box Piper, accessed October 16, 2025, <https://www.boxpiper.com/posts/how-poker-players-use-tech-tools-to-analyze-improve-and-win>
55. Blackjack 21: Strategy Trainer on the App Store, accessed October 16, 2025, <https://apps.apple.com/us/app/blackjack-21-strategy-trainer/id597239290>
56. Blackjack Trainer: All in one - Apps on Google Play, accessed October 16, 2025, <https://play.google.com/store/apps/details?id=com.bluewind.blackjacktrainer>

57. How to Record your Screen in Python, accessed October 16, 2025, <https://thepythoncode.com/article/make-screen-recorder-python>
58. opencv - Capture video data from screen in Python - Stack Overflow, accessed October 16, 2025, <https://stackoverflow.com/questions/35097837/capture-video-data-from-screen-in-python>
59. Create a Screen recorder using Python - GeeksforGeeks, accessed October 16, 2025, <https://www.geeksforgeeks.org/python/create-a-screen-recorder-using-python/>
60. How can vision AI models be used when playing cards - Ultralytics, accessed October 16, 2025, <https://www.ultralytics.com/blog/using-a-vision-ai-model-to-recognize-playing-cards>
61. Having Fun with YOLOv8: How Good Your Model in Detecting Playing Card? - Medium, accessed October 16, 2025, <https://medium.com/@sdwiulfah/having-fun-with-yolov8-how-good-your-model-in-detecting-playing-card-a468a02e4775>
62. Training a Custom Object Detection Model with YOLOv5 - Forecr.io, accessed October 16, 2025, <https://www.forecr.io/blogs/ai-algorithms/training-a-custom-object-detection-model-with-yolov5>
63. Python Optical Character Recognition (OCR): A Tutorial - Built In, accessed October 16, 2025, <https://builtin.com/data-science/python-ocr>
64. OCR Video Game Text : r/learnpython - Reddit, accessed October 16, 2025, [https://www.reddit.com/r/learnpython/comments/lyix7c/ocr\\_video\\_game\\_text/](https://www.reddit.com/r/learnpython/comments/lyix7c/ocr_video_game_text/)
65. Top 8 OCR Libraries in Python to Extract Text from Image - Analytics Vidhya, accessed October 16, 2025, <https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/>
66. Python Web Scraping Tutorial: Step-By-Step (2025) - Oxylabs, accessed October 16, 2025, <https://oxylabs.io/blog/python-web-scraping>
67. EdjeElectronics/OpenCV-Playing-Card-Detector: Python ... - GitHub, accessed October 16, 2025, <https://github.com/EdjeElectronics/OpenCV-Playing-Card-Detector>
68. Blackjack with Basic Strategy. - GitHub, accessed October 16, 2025, <https://github.com/v/blackjack>
69. tukiains/blackjack-gui: Blackjack with a graphical user interface - GitHub, accessed October 16, 2025, <https://github.com/tukiains/blackjack-gui>
70. Write an SRS document: How-tos, templates, and tips - Canva, accessed October 16, 2025, <https://www.canva.com/docs/srs-document/>
71. FREE Software Requirements Specification (SRS) Generator - PaceAI, accessed October 16, 2025, <https://paceai.co/tools/requirements-specification-generator/>
72. AI Software Requirements Specification - Bit.ai, accessed October 16, 2025, <https://bit.ai/ai-prompts/software-requirements-specification>
73. Software Requirement Specification (SRS) For Development of Training Database Platform, accessed October 16, 2025,



[https://doict.gov.bd/sites/default/files/files/doict.portal.gov.bd/notices/dd46d416\\_8ade\\_451a\\_8121\\_cc8bd56f83ad/2022-01-09-04-14-e5d865e195acba3dc0f70de53c14ac78.pdf](https://doict.gov.bd/sites/default/files/files/doict.portal.gov.bd/notices/dd46d416_8ade_451a_8121_cc8bd56f83ad/2022-01-09-04-14-e5d865e195acba3dc0f70de53c14ac78.pdf)

74. Srs | PDF - Slideshare, accessed October 16, 2025,  
<https://www.slideshare.net/slideshow/srs-238326687/238326687>
75. How to write a good SRS for your Project - GeeksforGeeks, accessed October 16, 2025,  
<https://www.geeksforgeeks.org/software-engineering/how-to-write-a-good-srs-for-your-project/>
76. basantrauniyar15/Software-Requirement-Specification-Document-Analysis-and-Parameter-Identification-Using-ML - GitHub, accessed October 16, 2025,  
[https://github.com/basantrauniyar15/Software-Requirement-Specification-Docu](https://github.com/basantrauniyar15/Software-Requirement-Specification-Document-Analysis-and-Parameter-Identification-Using-ML)  
[ment-Analysis-and-Parameter-Identification-Using-ML](https://github.com/basantrauniyar15/Software-Requirement-Specification-Document-Analysis-and-Parameter-Identification-Using-ML)
77. Third Party Tools and Services Policy - PokerStars, accessed October 16, 2025,  
<https://www.pokerstars.com/poker/room/prohibited/>
78. PokerStars Prohibited Software - 2025 Online Poker Rakeback Deals, accessed October 16, 2025, <https://www.rakeback.com/pokerstars/software/prohibited/>
79. 888poker Bans Accounts, Confiscates Balances - AI Involved? | Professional RakeBack, accessed October 16, 2025,  
<https://professionalrakeback.com/888-banning-poker-players-for-using-artificial-intelligence>
80. 888poker Pays \$250K Out of Own Pocket to Players Cheated by Bots and RTA Users, accessed October 16, 2025,  
[https://www.pokernews.com/news/2024/01/888poker-pays-250k-out-of-own-po](https://www.pokernews.com/news/2024/01/888poker-pays-250k-out-of-own-pocket-to-players-cheated-45279.htm)  
[cket-to-players-cheated-45279.htm](https://www.pokernews.com/news/2024/01/888poker-pays-250k-out-of-own-pocket-to-players-cheated-45279.htm)
81. What is Real Time Assistance (RTA)? - GGPoker Help, accessed October 16, 2025,  
<https://help.ggpoker.com/article/What-is-Real-Time-Assistance-RTA>
82. Online Poker Security Ecology Policy - GGPoker, accessed October 16, 2025,  
<https://ggpoker.com/network/security-ecology-policy/>
83. GGPoker Bans 31 Players for Illegal RTA Use - Gambling News, accessed October 16, 2025,  
[https://www.gamblingnews.com/news/ggpoker-bans-31-players-for-illegal-rta-u](https://www.gamblingnews.com/news/ggpoker-bans-31-players-for-illegal-rta-use/)  
[se/](https://www.gamblingnews.com/news/ggpoker-bans-31-players-for-illegal-rta-use/)
84. Penalties for Security & Ecology Policy Breaches - GGPoker Help, accessed October 16, 2025,  
<https://help.ggpoker.com/article/Penalties-for-Security-Ecology-Policy-Breaches>
85. Fairer Place to Play partypoker, accessed October 16, 2025,  
<https://www.partypoker.com/en/poker/p/safer-poker>
86. Partypoker tighten up restrictions on third-party tools - Poker.org, accessed October 16, 2025,  
<https://www.poker.org/latest-news/partypoker-tighten-up-restrictions-on-third-party-tools-aT7CE2G6doGz/>
87. Game Integrity – Find out more - PokerStars, accessed October 16, 2025,  
<https://www.pokerstars.com/poker/room/gameintegrity/faq/>
88. PartyPoker: Poker Cheaters Banned from Live and Online Play - PokerNews,

accessed October 16, 2025,

<https://www.pokernews.com/news/2025/04/partypoker-blacklist-cheaters-banned-48309.htm>

89. Is selling an online poker assistance software that is prohibited by poker sites legal?, accessed October 16, 2025,

<https://law.stackexchange.com/questions/47668/is-selling-an-online-poker-assistance-software-that-is-prohibited-by-poker-sites>

90. PokerStars™ – General Terms of Service, accessed October 16, 2025,

<https://www.pokerstars.com/tos/>